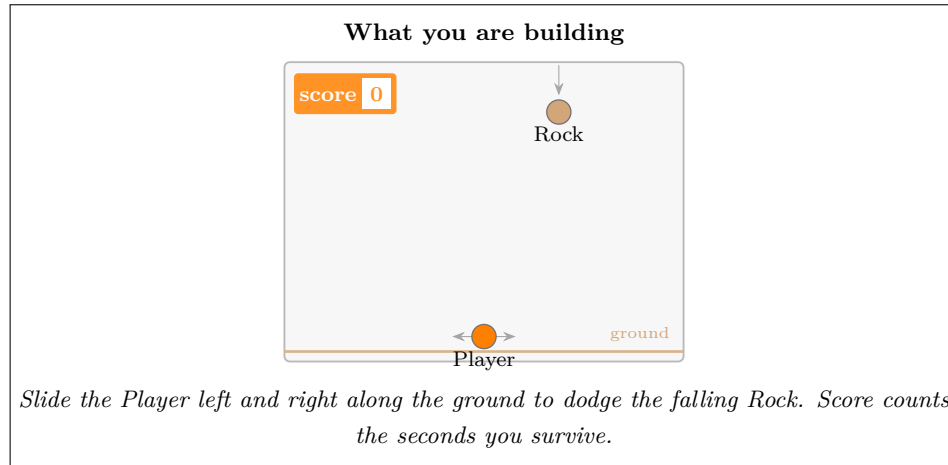


Name: _____

Date: _____

Welcome to Scratch F - Rock Dodge. One more game, built the same way as Coin Dash: most of it is given, and the grey ? **your turn** blocks are gaps you fill in. This time you dodge falling rocks and survive as long as you can. The trickiest gap uses an idea from Worksheet 16: a sprite's **y position** is the height of its *centre*, so when that number drops below the bottom of the stage, the rock has fallen off. Open scratch.mit.edu, click *Create*, and let us build it.



The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.
Operators	(green)	Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or.
Variables	(dark orange)	Remember a value like score: set, change, and show a variable.

Project 1: Move left and right

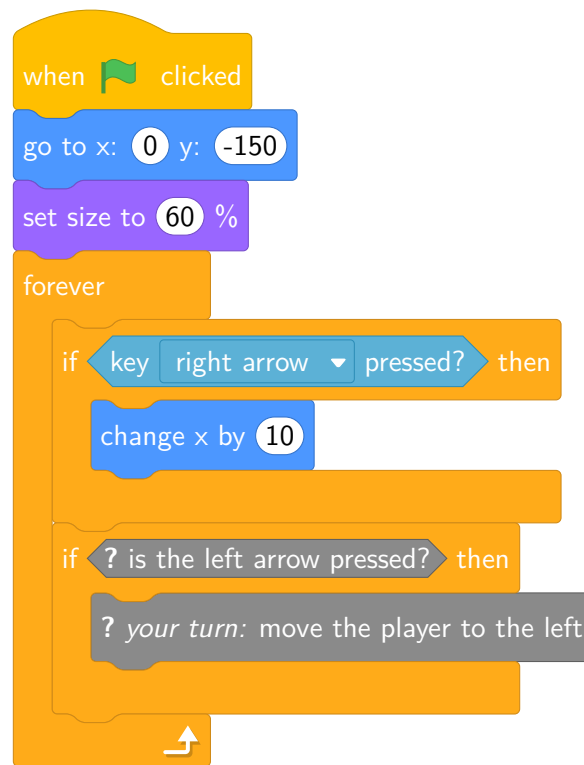
Put the player on the ground and let it slide left and right with the arrow keys. It does not jump. It just dodges along the bottom.

Get ready. Rename the cat sprite to **Player**. You will keep it near the bottom of the stage, where it can dodge the falling rocks.

Build it.

1. On the **Player**, start with **when green flag clicked**, then **go to x: 0 y: -150** (down on the ground) and **set size to (60) %**.
2. Add a **forever** loop. Inside it, the right arrow is given in full: **if <key [right arrow] pressed?>** then **change x by (10)**.
3. Below it is a second **if** shell for the left arrow. Both blanks are ? **your turn**: fill the grey hexagon with the left-arrow test, and the grey block with the move. Copy the pattern from the right arrow, but the other way.
4. Green flag, and slide left and right along the ground.

The Player's movement loop. The right arrow is done; fill the two grey gaps in the left-arrow **if**:



Player: dodge left and right

The player only ever changes x, so its height never changes. It stays locked to the ground line at y: -150 all game.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Change both **10** and **-10** to move faster or slower. Pick a speed that feels fair against the rocks.
2. Keep the player on screen: the stage edge is x: 240, so it cannot run off. Try **go to x: -150 y: -150** at the start to begin on the left.

Project 2: Make the rock fall

Add a rock that starts at the top at a random spot, falls straight down, and jumps back to the top once it drops off the bottom. That is one rock falling again and again, forever.

Get ready. Add a **Rock** sprite (click *Choose a Sprite*, pick a rock, an apple, or a ball, and rename it **Rock**). Make it small so it fits the stage.

New blocks.

y position

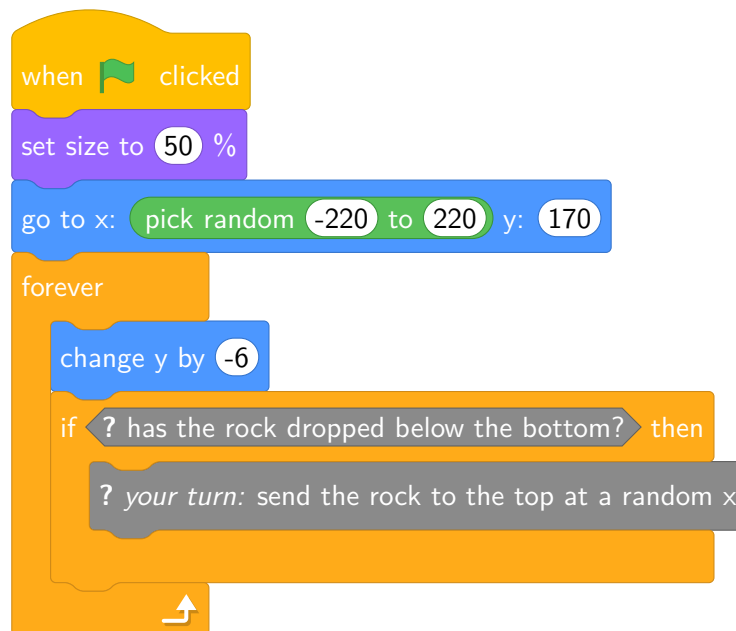
A blue Motion reporter: the height of the sprite's centre. It is a plus number near the top and a minus number near the bottom.

if <(y position) < (-170)>

True when the rock's centre has dropped below the bottom edge of the stage.

Build it.

1. On the **Rock**, start with **when green flag clicked**, then **set size to (50) %**.
 2. Add **go to x: (pick random (-220) to (220)) y: (170)**. That drops the rock in at a random column, up at the top.
 3. Add a **forever** loop. Its first block is given: **change y by (-6)**, which makes the rock fall.
 4. The next block is an **if** shell, with two **? your turn** gaps. The grey hexagon is the test: has the rock fallen off the bottom? Think about the **y position** block from Worksheet 16. The grey block inside is the action: send the rock back to the top at a new random column.
 5. Green flag. The rock falls, and the moment it leaves the bottom it reappears up top somewhere new.
- The Rock's falling loop. Fill the **if**: its test (the grey hexagon) and its action (the grey block):



Rock: fall and reset

The bottom edge is y: -180. Testing **y position < -170** resets the rock just as its centre slips past the bottom, so it never gets stuck off screen.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Make the rock fall faster: change `change y by (-6)` to a bigger drop like `-9`.
 2. Add a second rock: right-click the Rock and choose *duplicate*. The copy runs the same script, so now two rocks fall at once.
-

Project 3: Mini project: crash and score

Make it a real game. A `score` counts how many seconds you survive, and touching a rock ends the game. The longer you last, the higher you score.

Get ready. Make a variable `score` For all sprites. Both new scripts below go on the **Player**.

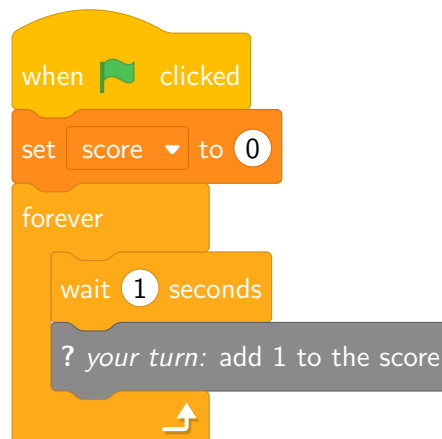
New blocks.

`if <touching [Rock]?>` True when the player is touching a rock. That is the crash.

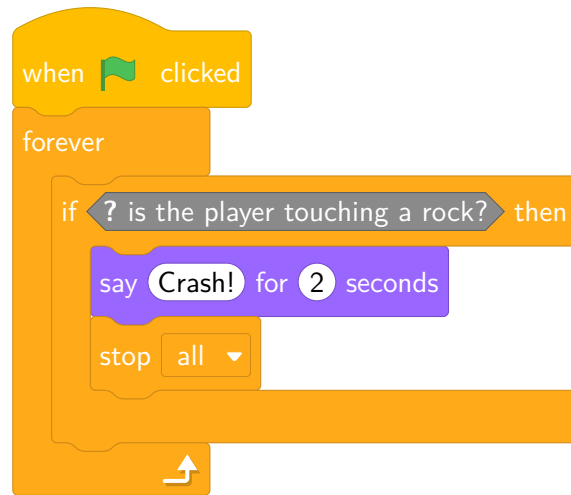
Build it.

1. On the **Player**, add a scoring script: `when green flag clicked`, then `set [score] to (0)`, then a `forever` loop whose first block is `wait (1) seconds`.
2. The next block is a `? your turn`: add one to the score on every tick, so the score counts the seconds you survive.
3. Add a second script for the crash: `when green flag clicked` then a `forever` loop.
4. Inside it is an `if` shell that ends the game. Its inside blocks (`say (Crash!)` and `stop [all]`) are given. The `? your turn` is the grey hexagon: fill in the test for touching a rock.
5. Green flag, and dodge. Your score climbs each second until a rock catches you.

The scoring loop, then the crash loop. Fill the grey stack block, and the grey hexagon in the `if`:



Player: one point per second



Player: game over on a hit

The score loop waits a second between points, but the crash loop has no wait, so it checks for a hit as fast as it can. That keeps the game fair: you never touch a rock without it counting.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Show the result: change `say (Crash!) for (2) seconds` to include the score, like `say (join (Score: ) (score))`.
 2. Give three lives instead of instant death. Make a variable `lives`, `set [lives] to (3)` at the start, and in the crash loop `change [lives] by (-1)`.
 3. End only when the lives run out: wrap the crash in `if <(lives) = (0)> then stop [all]`.
-